# Synergistic Motion Planning under Temporal Logic Specifications in 2D and 3D Workspaces

Yusif Razzaq
*ASEN Masters Student*
*University of Colorado Boulder*
yusif.razzaq@colorado.edu

Submission to: Dr. Morteza Lahijanian, University of Colorado Boulder, ASEN 6519: Hybrid Systems, Ann and H.J. Smead Aerospace Engineering Sciences College of Engineering and Applied Sciences.

## I. Introduction

For many robotics applications, it may be important to satisfy complex mission specifications while ensuring safe and feasible movements. This project aims to explore the implementation of a synergistic planner to develop dynamically constrained motion plans that satisfy a given temporal logic specification. This iterative framework utilizes a layered approach consisting of a high-level planner tasked with enforcing satisfaction of the mission specifications, and a low-level planner employing A-to-B motion-planning techniques to generate feasible transitions that follow the high-level plan. This approach was tested using various dynamic models in both 2D and 3D environments.

## II. Preliminaries

### A. Robot Modeling

To implement this planning approach, any robot can be modeled using its specific dynamics model. Generically, a robot and its environment can be defined as follows:

- $\mathcal{X} \subseteq \mathbb{R}^n$ is the state space of the robot, where each state $x \in \mathcal{X}$ represents a configuration of the robot in its environment.
- $\mathcal{O}$ is the obstacle space, representing the set of all obstacles present in the environment. Each obstacle is a convex polytope defined by its vertices.
- $\mathcal{F}$ is the free space and is defined as the complement of the obstacle space within the state space, i.e., $\mathcal{F} = \mathcal{X} \backslash \mathcal{O}$.
- $\mathcal{U} \subseteq \mathbb{R}^m$ is the control space of the robot, representing the set of all possible control inputs that the robot can apply.
- Let $f : \mathcal{X} \times \mathcal{U} \to \mathcal{X}$ denote the dynamics function, where $f(x, u)$ yields the next state of the robot given its current state $x$ and control input $u$.

### B. LTL Automaton Representation

Linear Temporal Logic (LTL) serves as an expressive language for specifying and verifying complex robotic behaviors. Unlike traditional motion planning approaches that focus solely on geometric constraints and spatial relationships, LTL enables representation of high-level mission requirements, temporal constraints, and logical relationships among events and states. A particularly useful representation of an LTL formula comes in the form of a finite automaton. Although techniques exist for reasoning about a large class of LTL formulas, this project will consider only a subset, known as co-safe LTL. Formulas of this syntax can be satisfied with finite words and can always be represented as a deterministic finite automaton (DFA). Let a DFA encoding the behaviors of an LTL formula be defined as:

$$\mathcal{A}_\varphi = (Q, \Sigma, \delta_\varphi, q_0, F)$$

- $Q = \{q_0, q_1, \ldots, q_n\}$ are a finite set of states,
- $\Pi = \{\pi_0, \pi_1, \ldots, \pi_n\}$ is the set of unique atomic propositions in the LTL formula,
- $\Sigma = 2^\Pi$ is the finite alphabet that enables transitions upon satisfaction of atomic propositions,
- $\delta_\varphi : Q \times \Sigma \to Q$ is a transition function that maps each state and input symbol to a unique successor state,
- $q_0 \in Q$ is the initial state,
- and $Q_{acc} \subseteq Q$ is the set of accepting states.

A word is accepted by the automaton $A_\varphi$ if an accepting state is reached under its symbols. These symbols represent satisfied propositions that, in the proper temporal arrangement, will satisfy the desired LTL specifications.

### C. Workspace Decomposition

To ensure compliance with the LTL specifications, it is important to have an abstract representation of the state space that allows for reasoning over finite discrete regions. To achieve a small yet sufficiently descriptive abstraction, a procedure for workspace decomposition is used. For this representation to maintain an observational equivalence with the underlying continuous state space, the workspace regions must respect the boundaries of regions that do not share ob-

servations. A region serves as an equivalent class, containing all adjacent states $x \in \mathcal{X}$ that share observations.

Since the specifications may be satisfied when a subset of the states attains a particular configuration, the workspace decomposition can be performed in a lower-dimensional space that is minimally expressive enough to capture the regions defined by the specifications. For example, if the robot is only required to reach a 2D region of the environment, it suffices to use a 2D projection of the full state space, in which only two state elements must satisfy the constraints. For a 3D region, a prismatic decomposition is required. Yet, higher-dimensional regions may be necessary if an additional parameter (e.g., the quadrotor's battery level) must meet a certain threshold as it enters the volume. Formally, we can define an abstraction as follows:

- $\mathcal{W} \subset \mathbb{R}^{n_w}$ is the bounded workspace of the robot where $n_w$ can capture all components of the state necessary to satisfy the specifications.
- $h : \mathcal{X} \rightarrow \mathcal{W}$ is a mapping function that converts a continuous state to the corresponding point in the workspace, i.e., $h(x) = \text{proj}_{n_w}(x)$.
- $L : \mathcal{W} \rightarrow 2^{\Pi}$ is a labeling function that assigns every point of the workspace to a set of satisfied atomic propositions.
- $\mathcal{R} = \{r_i \subset \mathcal{W} \mid L(h(x)) = L(h(x')) \ \ \forall x, x' \in r_i\}$ is the set of decomposed regions of the workspace, representing observationally equivalent classes.
- $E = \{(r_i, r_j) \in \mathcal{R} \mid r_i, r_j \text{ are adjacent}\}$ is the set of bidirectional, connected edges between all adjacent regions.
- $\mathcal{D} = (\mathcal{R}, E)$ represents the abstraction graph consisting of all regions $r \in \mathcal{R}$ as nodes and edges in $E$.

### D. Product Automaton Construction

The construction of a product automaton involves combining the states, transitions, and acceptance conditions of the two constituent automata to create a new automaton that captures the joint behavior of both systems. Here, the abstraction graph that connects adjacent regions and the LTL automaton that encodes mission specifications can be combined to derive a product automaton in hybrid space. This enables the development of high-level plans that, if dynamically feasible, will ensure the requirements are met. Let the product automaton between $\mathcal{D}$ and $\mathcal{A}_\varphi$ be defined as a tuple:

$$\mathcal{A}_P = (P, P_0, \delta_P, P_{acc})$$

- $P = Q \times \mathcal{R}$ are the hybrid state of the system.
- $P_0 = \{(q_0, r_i) \ \forall r_i \in \mathcal{R}\}$ is the set of initial states.
- $\delta_P : P \times \Sigma \rightarrow P$ is a deterministic transition function such that $(p.r,\ p'.r) \in E$ and $p.q' = \delta_\varphi(p.q,\ L(p.r))$.
- $P_{acc} = \{(q, r) \mid q \in Q_{acc}\}$ is the set of accepting hybrid states.

Once this graph is constructed, the initial state of the robot can be related to a specific hybrid state, $p = (q_0, r) \mid h(x_0) \in r$. Then, Dijkstra's graph search algorithm can be used to find the shortest path to the set of accepting hybrid states.

### E. Search Tree

The low-level planner aims to extend the search tree according to the regions specified by the high-level plan using sampling-based techniques. This utilizes Rapidly-Exploring Random Trees (RRT) with kinodynamic constraints [1], which involves integrating the robot's dynamics model into the RRT algorithm to ensure that generated paths are feasible as it explores the configuration space. As feasible trajectories are sampled and extensions are made, a tree structure is constructed:

- $V = (q, x, u, \tau)$ is the set of vertices in the search tree, consisting of the mode $q$, the continuous state $x$, the control input $u$ applied, and the time duration $\tau$ from the previous vertex.
- $\mathcal{E} \subset \{(v_i, v_j) \mid \exists \tau \in \mathbb{R}_+ \text{ and } \exists u \in \mathcal{U} \ s.t. \ v.x_j = v.x_i + \int_0^\tau f(x, u)\, d\tau\}$ are the set of edges that connect pairs of vertices and represent dynamically feasible transitions between configurations. The directional edge $e_{ij}$ connects vertex $v_i$ to vertex $v_j$.
- $\mathcal{T} = (V, \mathcal{E})$, is the search tree containing the vertices and edges of the RRT. Each vertex in this tree has a unique path to the root node, representing a trajectory through hybrid space.
- $v_{init} = (q_0, x_0, \cdot, \cdot)$ is the initial configuration from which the tree expansion begins.
- $v = v_{goal} \iff h(v.x) \in r_i$ is a vertex in the region defined by the high-level planner. The tree expansion process continues until a goal vertex is found, or until a specified termination condition is met.

## III. Problem Formulation

We consider two scenarios in which a robot operates within a finite workspace characterized by known obstacles. The robot's dynamics model is known, allowing for open-loop control strategies to effect its motion through the environment. Additionally, the workspace contains several regions of interest that must be visited according to temporal logic specifications, formulated in co-safe LTL syntax. Our objective is to generate a motion plan for the robot that meets these specifications and ensures dynamic feasibility.

### A. Reeds-Shepp Car Robot

In the first phase, we consider a robot with a simple car model operating in a 2D workspace. The robot is modeled as a Reeds-Shepp car with the two front wheels used for steering and the two rear wheels used for propulsion. A state space

dynamics model was used to describe the motion of the robot following the derivation here [2]:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ (v/L)\tan\phi \\ u_1 \\ u_2 \end{bmatrix}$$

where $\dot{\mathbf{x}}$ represents the time derivative of the state, x, and u describes the input controls. The state and control vectors are defined as

$$\mathbf{x} = [x, y, \theta, v, \phi]^T$$

$$\mathbf{u} = [u_1, u_2]^T$$

and the individual components of the state and control space observe the following constraints:

- positions $h(\mathbf{x}) = (x, y) \in \mathcal{W}$,
- orientation $\theta \in S$,
- linear speed $v \in [-1/6, 1/2]$,
- steering angle $\phi \in [-\pi/4, \pi/4]$,
- linear acceleration $u_1 \in [-1/2, 1/2]$,
- and steering angle rate $u_2 \in [-\pi/4, \pi/4]$.

The workspace $\mathcal{W}_1$ with $n_w = 2$, shown in Figure 1, contains three regions of interest, $R_a$, $R_b$, and $R_c$. The teal regions denote regions $R_a$ and $R_b$, while the light green region represents the final goal, $R_c$. The temporal logic specifications considered for this problem are "eventually visit all regions, but must visit $a$ and $b$ before $c$". Formally,

$$\varphi = Fa \wedge Fb \wedge Fc \wedge \neg cUa \wedge \neg cUb$$

### B. Quadcopter Robot

The second robot considered in this work is a quadcopter operating within a 3D workspace. This design consists of four identical, symmetrically positioned rotors operating under hovering conditions. The rotors are slightly angled inwards, such that the quadcopter can move horizontally without performing attitude maneuvers. The linearized model about a steady hover operating condition is defined as follows:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -k_l & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -k_l & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -k_v \end{bmatrix} \quad B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -\delta & 0 & \delta & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -\delta & 0 & \delta \\ 0 & 0 & 0 & 0 \\ \gamma & \gamma & \gamma & \gamma \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & y & \dot{y} & z & \dot{z} \end{bmatrix}^T$$

$$\mathbf{u} = \begin{bmatrix} T_1 & T_2 & T_3 & T_4 \end{bmatrix}^T$$

where $k_l$ and $k_v$ are the specific drag coefficients in the horizontal and vertical directions, and $\delta$, $\gamma$ are angles that defined the tilt of the rotors. There are four controls corresponding to the specific thrust of each rotor $T_i$.

The workspace $\mathcal{W}_2$ with $n_w = 3$ similarly contains three regions of interest, $R_a$, $R_b$, and $R_c$. The same temporal logic specifications were considered for this case as in $\mathcal{W}_1$. Conversely, the mapping function here uses the three positional components of the state: $h(\mathbf{x}) = (x, y, z)$.

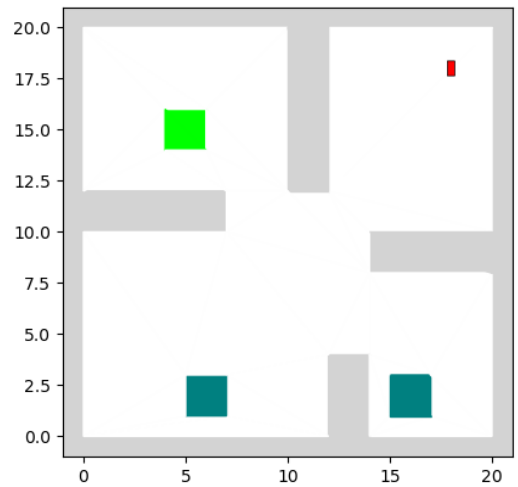


Figure 1. Workspace $\mathcal{W}_1$ with regions $a$ and $b$ in teal, region $c$ in green, and initial robot position in red.

## IV. Approach

### A. LTL Automaton Representation

For both robots, the same co-safe LTL specification was defined over the set of atomic propositions $\Pi = \{a, b, c\}$. These propositions are satisfied when the positional elements of the robot are contained within the corresponding regions $x \models \pi_i \iff h(x) \in R_i$. The formula $\varphi$ can be represented as a DFA as shown in Figure 2, omitting self-transitions and assuming only singleton symbols will be observed $\Sigma = \{a, b, c\}$. The state is always initialized at $q_0$, and there exist two distinct traces that reach the accepting state $q_4$.

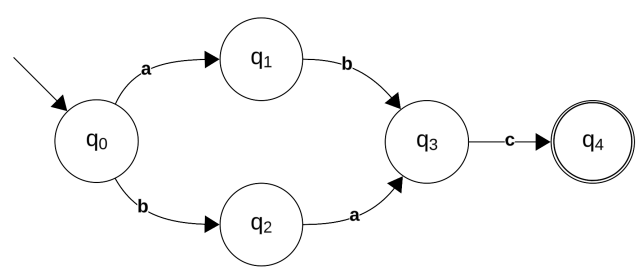


Figure 2. Automaton representation of the LTL formula $\varphi = Fa \wedge Fb \wedge Fc \wedge \neg cUa \wedge \neg cUb$. $q_0$ and $q_4$ are the initial and accepting states, respectively.

### B. Workspace Decomposition

The workspace was decomposed into regions based on the workspace boundaries, obstacles, and regions of interest. Algorithm 1 details the steps used to ensure the triangulation process respects the various regions of the workspace. Figures 3 and 4 show the resulting regions produced by this decomposition procedure, where empty regions are labeled white, obstacles in grey, regions $a$ and $b$ in teal, and the goal region $c$ in green. In the case of $\mathcal{W}_1$, each region is represented by a triangle, while in $\mathcal{W}_2$ the regions are defined as tetrahedra, computed using a constrained Delaunay triangulation [3]. In both cases, adjacency was used to determine connected nodes in the abstraction graph, and labeling was performed by observing the labels of each region's centroid.

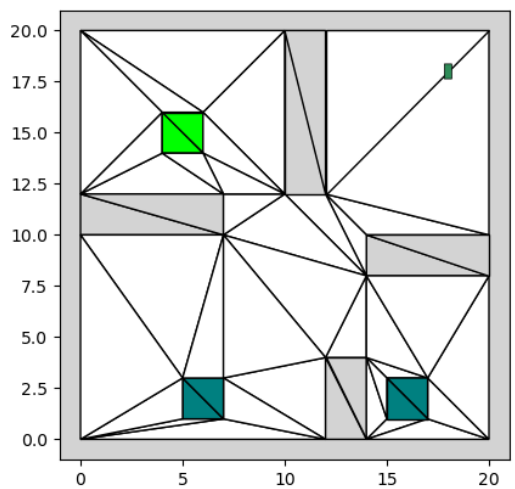


Figure 3. 2D decomposition of $\mathcal{W}_1$ using Constrained Delaunay Triangulation.

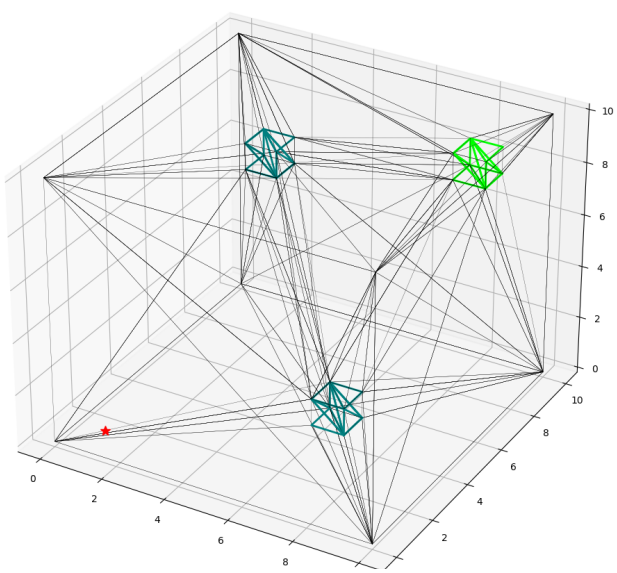

Figure 4. 3D decomposition of $\mathcal{W}_2$ with initial robot position (red).

**Algorithm 1** : Workspace Decomposition

**Input** Set of obstacles $\mathcal{O}$, set of regions of interest $R$
**Output** Decomposed workspace with labeled regions
Initialize an empty set of constraints $C$
**for** each obstacle $o_i \in \mathcal{O}$ **do**
  **for** each edge $e_j$ in $o_i$ **do**
    $C \leftarrow C \cup \{e_j\}$
  **end for**
**end for**
**for** each region of interest $r_i \in R$ **do**
  **for** each edge $e_j$ in $r_i$ **do**
    $C \leftarrow C \cup \{e_j\}$
  **end for**
**end for**
$CDT \leftarrow$ `computeDelaunayTriangulation`$(\mathcal{W}, C)$
Initialize an empty set of labeled regions $\mathcal{R}$
**for** each triangle $t_i \in CDT$ **do**
  $c_i \leftarrow$ `getCentroid`$(t_i)$
  **if** $c_i \in r_n$ **then** assign $L(r_n)$ to triangle $t_i$
  **end if**
  $\mathcal{R} \leftarrow \mathcal{R} \cup \{t_i\}$
**end for**
**Return** $\mathcal{R}$

### C. High Level Planner

Once representations of the LTL formula and the decomposed workspace were defined, a product automaton was constructed according to Algorithm 2. This creates a graph structure with edge costs, which will be subsequently used to find a feasible path through the hybrid space that satisfies the LTL formula. The `graphSearch()` method performs Dijkstra's algorithm to find the shortest path from the initial state to the set of accepting states. The resulting high-level plan will consist of a sequence of nodes from this graph that the low-level planner must visit in order.

**Algorithm 2** Construct Product Automaton

**Input:** $A_\varphi = (Q, \Sigma, \delta_\varphi, q_0, Q_{acc})$, $D = (\mathcal{R}, E)$
**Output:** Product Automaton $\mathcal{A}_P = (P, P_0, \delta_P, P_{acc})$
$P = Q \times \mathcal{R}$
$P_0, \delta_P, P_{acc} \leftarrow \{\}$
**for** each hybrid state $p_i = (q_i, r_i) \in P$ **do**
  **if** $q_i = q_0$ **then** $P_0 \leftarrow P_0 \cup \{p_i\}$
  **end if**
  **if** $q_i \in Q_{acc}$ **then** $P_{acc} \leftarrow P_{acc} \cup \{p_i\}$
  **end if**
  $q_j \leftarrow \delta_\varphi(q_i, L(r_i))$
  **for** each edge $e_i = (r_i, r_j) \in E$ **do**
    $\delta_P \leftarrow \delta_P \cup (p_i, (q_j, r_j), cost = 1)$
  **end for**
**end for**
**Return** $\mathcal{A}_P$

### D. Low Level Planner

With a defined high-level plan, each consecutive pair of hybrid states is passed to the low-level planner as initial and goal regions. The goal region will be defined as the terminating condition of the RRT search. A random sample, $q_{rand}$, from the state space, with a goal bias sampled from the goal region, is taken from a uniform distribution and serves as the desired point to extend towards. Then, multiple random controls are sampled uniformly from the control space in addition to a control duration, $\Delta t \sim \mathcal{U}(0.5, 1.5)$. A Runge-Kutta integrator then propagates the state according to the robot's dynamics under the applied control. From the sampled controls, the resulting state that is closest to the sampled point, $q_{rand}$, will be chosen for extension. The distance metric utilized here was the Euclidean distance, considering only the positional elements of the state.

Once a node of the search tree enters the goal region, the subroutine would terminate, and the next search would commence, initialized at the previous goal node and extending towards the next goal region in the high-level plan. There is a finite number of samples that each subroutine is allotted before terminating with a failure. This limit on the number of samples is incrementally increased for every iteration between the two planning layers to maintain probabilistic completeness.

### E. Synergistic Planner

The synergistic planner consists of the two previously described layers and closely follows the implementation introduced here [4]. Initially, the high-level planner identifies the most cost-effective plan, which is subsequently forwarded to the low-level planner. This planner will attempt to expand the search tree from the current state towards the subsequent state outlined in the high-level plan. The low-level planner records the number of attempts made to transition between two hybrid states, which is then used to update the edge cost for that transition. The following expression determines the cost calculation:

$$c(p_i, p_j) = \frac{\texttt{numSamples}(p_i, p_j)}{\texttt{volume}(p.r_j) \cdot \texttt{distToAcc}(p_j)}$$

where the `numSamples` is a cumulative counter of the sampled points used to attempt a transition by the RRT planner, `volume` is the area or volume of the successor region, and `distToAcc` is the minimum number of edges needed to reach an accepting state. This allows the algorithm to effectively learn the transition probabilities between regions in the workspace and better guide its subsequent attempts to find a feasible motion plan. If no complete solution is found in the allotted attempts, a new high-level plan will be derived using the updated costs. Figure 5 shows the system architecture, while Algorithm 3 outlines the steps used in the implementation of this approach.

**Algorithm 3** Synergistic Motion Planner

**Input:** $\mathcal{A}_P = (P, P_0, \delta_P, P_{acc})$, $RRT = (\mathcal{X}, \mathcal{O}, \mathcal{U}, f(x, u))$
**Output:** Motion Plan $\sigma_x = \{(x_0, u, \tau), \dots\}$

$p_0 \leftarrow (q_0,\ \texttt{locateRegion}(h(x_0)))$
$i \leftarrow 1$
**while** $i < maxIterations$ **do**
  $\sigma_x \leftarrow \{\}$
  $\sigma_{HL} = \texttt{graphSearch}(\mathcal{A}_P, p_0)$
  **for** $p_{HL} \in \sigma_{HL}$ **do**
    **if** first **then** $x_{init} \leftarrow x_0$
    **end if**
    $\sigma_{sub}$, attempts $\leftarrow RRT.\texttt{Plan}(x_{init}, p.r_{HL}, 1000 \cdot i)$
    $\mathcal{A}_P \leftarrow \texttt{updateWeights}(\mathcal{A}_P, \text{attempts})$
    **if** $\sigma_{sub} \neq \{\}$ **then**
      $\sigma_x \leftarrow \sigma_x \cup \sigma_{sub}$
    **else** break
    **end if**
    $x_{init} \leftarrow \sigma_{sub}.last()$
  **end for**
  $i \leftarrow i + 1$
**end while**
**Return** $\sigma_x$

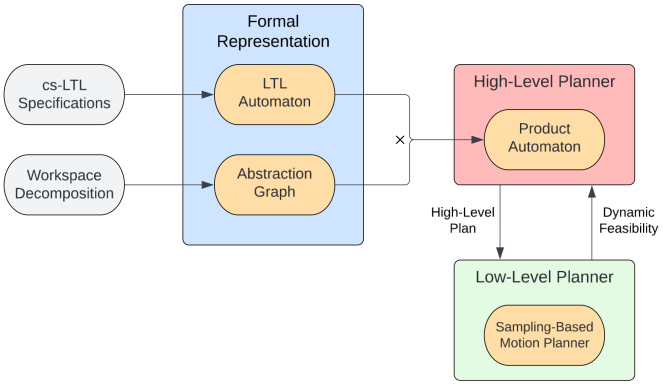


Figure 5. Synergistic motion-planning framework.

## V. Results

### A. Reeds-Shepp Car Results

The implementation of this approach demonstrated a reliable method for generating motion plans that satisfy the given LTL specifications while observing the robot's dynamic constraints. The planning time for the workplace considered would typically be several seconds. The motion plans resulting from two instances are depicted in Figure 6, where the LTL formula is satisfied by sequentially visiting $a$ and then $b$, and vice versa. However, the resulting trajectories are notably sub-optimal, as the low-level planner does not attempt to guide the search beyond reaching the desired region. In such cases, implementing an algorithm like Stable Sparse RRT (SST) [5] could yield more favorable plans.

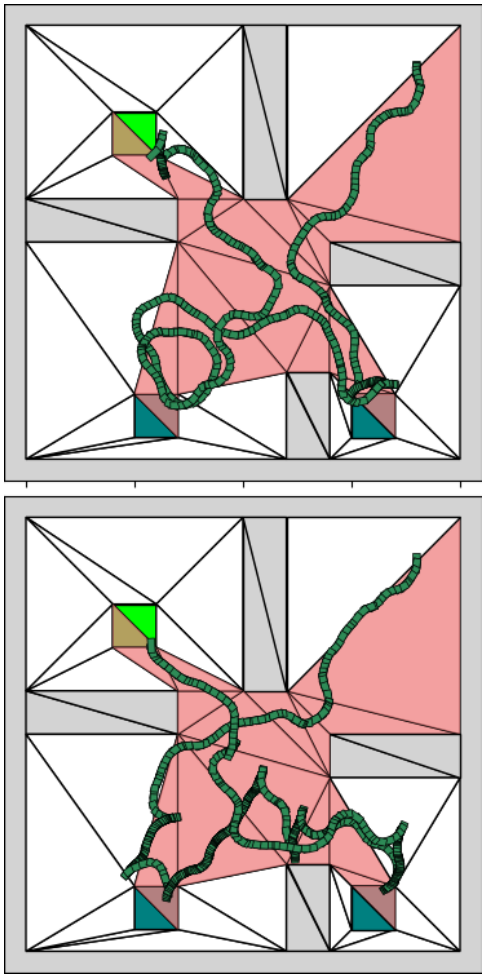

Figure 6. Motion plans for car-like robot and two trajectories satisfying $\varphi = Fa \wedge Fb \wedge Fc \wedge \neg cUa \wedge \neg cUb$. High-level plan shown in red.

### B. Quadrotor Results

This approach also solved the proposed problem in a 3D workspace using the quadrotor dynamics model, as depicted in Figure 7. As in the previous case, the quadrotor's trajectories are not optimal for similar reasons. The planning times in this environment were considerably longer, averaging about 5 minutes. This may be due to a more constrained dynamics model and planning complexity in the higher-dimensional state space. However, perhaps most consequential, were the presence of volumetrically small regions in the workspace decomposition.

The traversal of these intermediating regions was required by the high-level planner, while only providing a narrow margin for the low-level planner to succeed. In future work, the conjunction of observationally equivalent adjacent regions will be considered to meet a minimum volume threshold, thereby improving planning time. Grouping such regions would also reduce the size of the abstraction graph and, consequently, the product automaton.

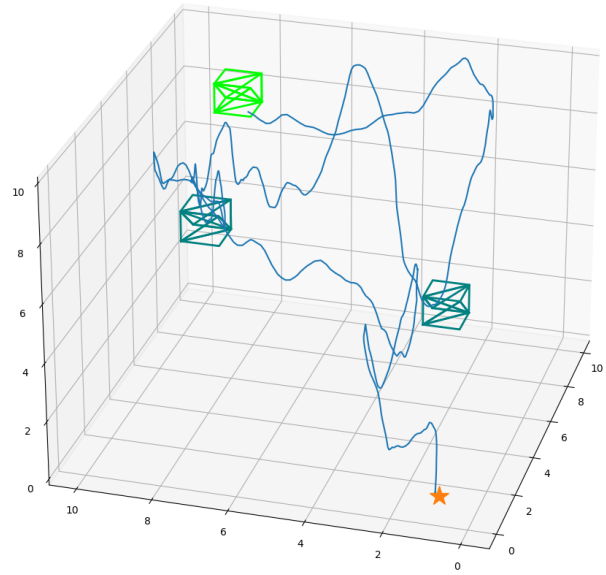


Figure 7. Motion plan for quadrotor satisfying the LTL formula: $\varphi = Fa \wedge Fb \wedge Fc \wedge \neg cUa \wedge \neg cUb$.

## VI. Conclusion

In this work, we explored an iterative approach to complex motion planning problems, aiming to satisfy complex mission specifications while ensuring safe and feasible movements. Using a synergistic planner comprising high- and low-level planning layers, we address scenarios involving a car-like robot operating in a 2D workspace and a quadcopter in a 3D workspace.

In the first scenario, the approach effectively generated motion plans that satisfied the given LTL specifications. However, the resulting trajectories were suboptimal due to the low-level planner's limited guidance beyond reaching the desired regions. For enhanced plans, the adoption of asymptotically optimal algorithms is suggested.

Similarly, the approach demonstrated success in having the quadcopter navigate a 3D workspace. However, longer planning times were observed in coincidence with notably small, observationally equivalent regions produced by the workspace decomposition. This is believed to have significantly impacted planning efficiency, suggesting future exploration into optimizing this aspect.

Overall, this approach proved robust and effective for solving complex path-planning challenges, with opportunities for further optimization and algorithmic enhancements to improve planning efficiency and trajectory optimality.

## References

[1] S. M. LaValle and J. J. Kuffner, Randomized kinodynamic planning, International Journal of Robotics and

Research, vol. 20, no. 5, pp. 378400, 2001.
[2] Morteza Lahijanian. Lecture 14: Kinodynamic motion planning. Presentation slides, Fall 2023. Accessed: November 30, 2023.
[3] The CGAL Project, "CGAL User and Reference Manual." CGAL Editorial Board, 2024.
[4] M. Lahijanian, M. R. Maly, D. Fried, L. E. Kavraki, H. Kress-Gazit and M. Y. Vardi, "Iterative Temporal Planning in Uncertain Environments With Partial Satisfaction Guarantees," in IEEE Transactions on Robotics, vol. 32, no. 3, pp. 583-599, June 2016, doi: 10.1109/TRO.2016.2544339.
[5] Y. Li, Z. Littlefield, K. E. Bekris, "Asymptotically Optimal Sampling-based Kinodynamic Planning," International Journal of Robotics Research, vol. 35, no. 5, pp. 528-564, 2016, doi:10.1177/0278364915614386.